Inteligentny planer nauki dla studentów

Poziomy I–II TRL – badania podstawowe i przemysłowe

---

1. Poziom I TRL – badania podstawowe

1.1. Opis idei

W wyniku analiz organizacji nauki studentów w okresie sesji egzaminacyjnej nasz zespół opracował koncepcję algorytmu, który automatycznie układa indywidualny harmonogram nauki. Algorytm uwzględnia ilość materiału do opanowania, terminy egzaminów, szacowaną trudność poszczególnych przedmiotów oraz czas, jakim student faktycznie dysponuje w danym dniu.

Zauważono, że większość studentów planuje naukę liniowo („tyle stron dziennie") i nie uwzględnia zjawiska zapominania. Wprowadzenie do algorytmu mechanizmu powtórek rozłożonych w czasie oraz ważenia przedmiotów według trudności i terminu pozwoliło we wstępnej symulacji na modelowych danych uzyskać plany, które:

- zmniejszały ilość nauki „na ostatnią chwilę" o ok. 30% względem planu liniowego,
- pozwalały zaplanować powtórkę całego materiału przed egzaminem w ok. 90% przypadków (przy planie liniowym ok. 60%).

Zauważono również, że algorytm można rozszerzyć o dynamiczną korektę. Po każdej sesji nauki student ocenia swoją znajomość tematu (np. krótkim quizem lub oceną w skali 1–5), a planer przesuwa zagadnienia słabiej opanowane na wcześniejsze terminy i zwiększa liczbę ich powtórek. Wstępna analiza wskazuje, że można w ten sposób przewidywać, które zagadnienia będą wymagały największej liczby powtórek.

2. Poziom II TRL – badania przemysłowe

2.1. Koncepcja technologii i jej przyszłego zastosowania

Przeprowadzono wstępną analizę możliwych zastosowań technologii algorytmu planowania nauki. Ustalono, że można ją rozwinąć i zastosować na większości platform, tj. w systemach Android i iOS oraz w przeglądarce internetowej (Windows, Linux, macOS). Optymalnym rozwiązaniem jest, aby do zaprojektowanego algorytmu stworzyć aplikację, która stanowić będzie interfejs dla użytkownika, za pomocą którego będzie on mógł wykonywać różne operacje związane z użyciem algorytmu, w szczególności:

- dodawać przedmioty, terminy egzaminów oraz zakres materiału,
- określać szacowaną trudność przedmiotów i dostępny czas nauki,
- przeglądać wygenerowany plan na dany dzień i tydzień,
- oznaczać postępy (np. ocena znajomości tematu w skali 1–5),
- otrzymywać automatycznie zaktualizowany plan i przypomnienia.

Aplikacja powinna być prosta i intuicyjna oraz zgodna z zasadą: im mniej kroków (kliknięć) do rezultatu, tym lepiej. Docelowo użytkownik powinien otrzymać pierwszy plan nauki po wprowadzeniu jedynie najważniejszych danych (przedmiot, termin egzaminu, ilość materiału).

Zauważono, że algorytm po odpowiedniej adaptacji może być zastosowany do przygotowania powtórek w postaci krótkich quizów i fiszek, których wyniki bezpośrednio zasilają planer i pozwalają dokładniej wskazywać zagadnienia wymagające najwięcej powtórek. Opcja modułu powtórek i quizów zostanie wykorzystana jako jeden z komponentów aplikacji produkcyjnej. Docelowo aplikacja będzie również korzystać z integracji z kalendarzem użytkownika oraz z powiadomień przypominających o zaplanowanej nauce.

Przyjęto, że projekt będzie się opierał na następujących komponentach:

- **silnik planowania** – algorytm układający harmonogram, wydzielony jako niezależna biblioteka lub usługa,
- **aplikacja mobilna i webowa** – prosty interfejs użytkownika,
- **moduł postępów i powtórek** – oceny, quizy, fiszki,
- **integracje** – kalendarz, powiadomienia, synchronizacja konta.

Na tym etapie przyjęte założenia nie zostały jeszcze potwierdzone dowodami ani szczegółowymi analizami.

2.2. Lista rzeczy potrzebnych do dalszego rozwoju projektu

Zweryfikowano, jakie zasoby zespół ma już dostępne, a jakie będą jeszcze potrzebne. Zgodnie z założeniem wymagania przyjęto na poziomie optymalnym.

Zasób	Przeznaczenie	Status
2x komputer stacjonarny lub wydajny laptop (min. 16 GB RAM, dysk SSD)	rozwój aplikacji i algorytmu	dostępne
dostęp światłowodowy do Internetu	praca zespołowa, chmura, repozytorium kodu	dostępne
2x środowisko programistyczne (np. Visual Studio Code lub IntelliJ, Android Studio)	tworzenie i testowanie kodu	dostępne
repozytorium kodu z automatycznymi testami (GitHub)	wersjonowanie, praca w zespole, ciągła integracja	dostępne
narzędzie do zarządzania projektem (CollaboryChat)	harmonogram i podział zadań	dostępne
środowisko do analizy danych (Python, Jupyter)	prototyp i strojenie algorytmu, symulacje planów	dostępne
środowisko do testów: smartfony	testy aplikacji na różnych urządzeniach	dostępne
serwer w chmurze z bazą danych (np. PostgreSQL)	backend, konta użytkowników, synchronizacja danych	do pozyskania
konta deweloperskie Google Play i Apple App Store	publikacja aplikacji	do pozyskania

Zasób	Przeznaczenie	Status
dane testowe (przykładowe, zanonimizowane plany sesji) oraz grupa testowa 10–20 studentów	weryfikacja algorytmu i użyteczności aplikacji	do pozyskania
polityka prywatności i zgodność z RODO	ochrona danych o postępach w nauce	do opracowania